Análisis de Caso

Implementación de Servicios de Mensajería Cloud

Eva Durán

alkemy

Análisis de Caso

Implementación de Servicios de Mensajería Cloud

Situación inicial 📍

Una empresa de comercio electrónico, E-ComExpress, está experimentando problemas de comunicación interna entre sus módulos de procesamiento de pedidos y notificación de envíos. El sistema monolítico actual genera cuellos de botella y errores en la transmisión de información, lo que retrasa la actualización del estado de los pedidos y afecta la satisfacción del cliente. Para resolver estos inconvenientes, la dirección tecnológica ha decidido implementar servicios de mensajería en la nube utilizando Amazon SQS y Amazon SNS, con el fin de desacoplar los procesos y mejorar la escalabilidad y tolerancia a fallos del sistema.

Descripción del Caso 🔍

Eres un ingeniero de la nube en E-ComExpress, encargado de liderar la implementación de servicios de mensajería cloud. Tu misión es diseñar y configurar un sistema que utilice Amazon SQS para la gestión asíncrona de mensajes entre módulos y Amazon SNS para la difusión de notificaciones en tiempo real a diversos servicios (por ejemplo, actualizaciones de pedidos y alertas a los equipos de logística). El desafío consiste en analizar la situación actual, identificar los puntos críticos de comunicación, y proponer una solución que optimice la integración de datos y la resiliencia del sistema.

Instrucciones 💡

1. **Análisis de la Situación:**
    - Describe brevemente los problemas que enfrenta el sistema actual y cómo afectan la operación y experiencia del cliente.
    - Explica la importancia de implementar servicios de mensajería para desacoplar procesos y mejorar la escalabilidad.
2. **Diseño de la Solución:**
    - Elabora un diagrama (opcional, utilizando AWS Academy o herramientas gratuitas de diagramación) que ilustre la integración de Amazon SQS y SNS en la arquitectura actual.
    - Describe el flujo de mensajes: cómo se envían, reciben y procesan a través de SQS, y cómo SNS distribuye las notificaciones a los suscriptores.

3. **Propuesta de Implementación:**

    - Explica los pasos básicos para configurar una cola en SQS y un tema en SNS.
    - Menciona cómo se gestionarán las políticas de acceso y la retención de mensajes.
    - Describe brevemente las ventajas de esta solución, tales como la tolerancia a fallos, la mejora en la integración de datos y la optimización de tiempos de respuesta.

4. **Retroalimentación y Evaluación:**

    - Reflexiona sobre los posibles desafíos o limitaciones que podrías encontrar al implementar estos servicios.
    - Propón medidas adicionales para mitigar posibles fallos en la comunicación.

Entregables

- Un reporte escrito o presentación que incluya:

    - Un análisis de la situación actual y los problemas detectados.

    - El diagrama de la solución propuesta (opcional).

    - La descripción de la solución, incluyendo la configuración de SQS y SNS, y los beneficios esperados.
    - Reflexiones sobre las limitaciones y medidas de mitigación.

- Archivos con ejemplos de configuración (si se realizaron simulaciones en AWS Academy).

DESARROLLO

1. Análisis de la situación actual

Problema del sistema actual y su impacto

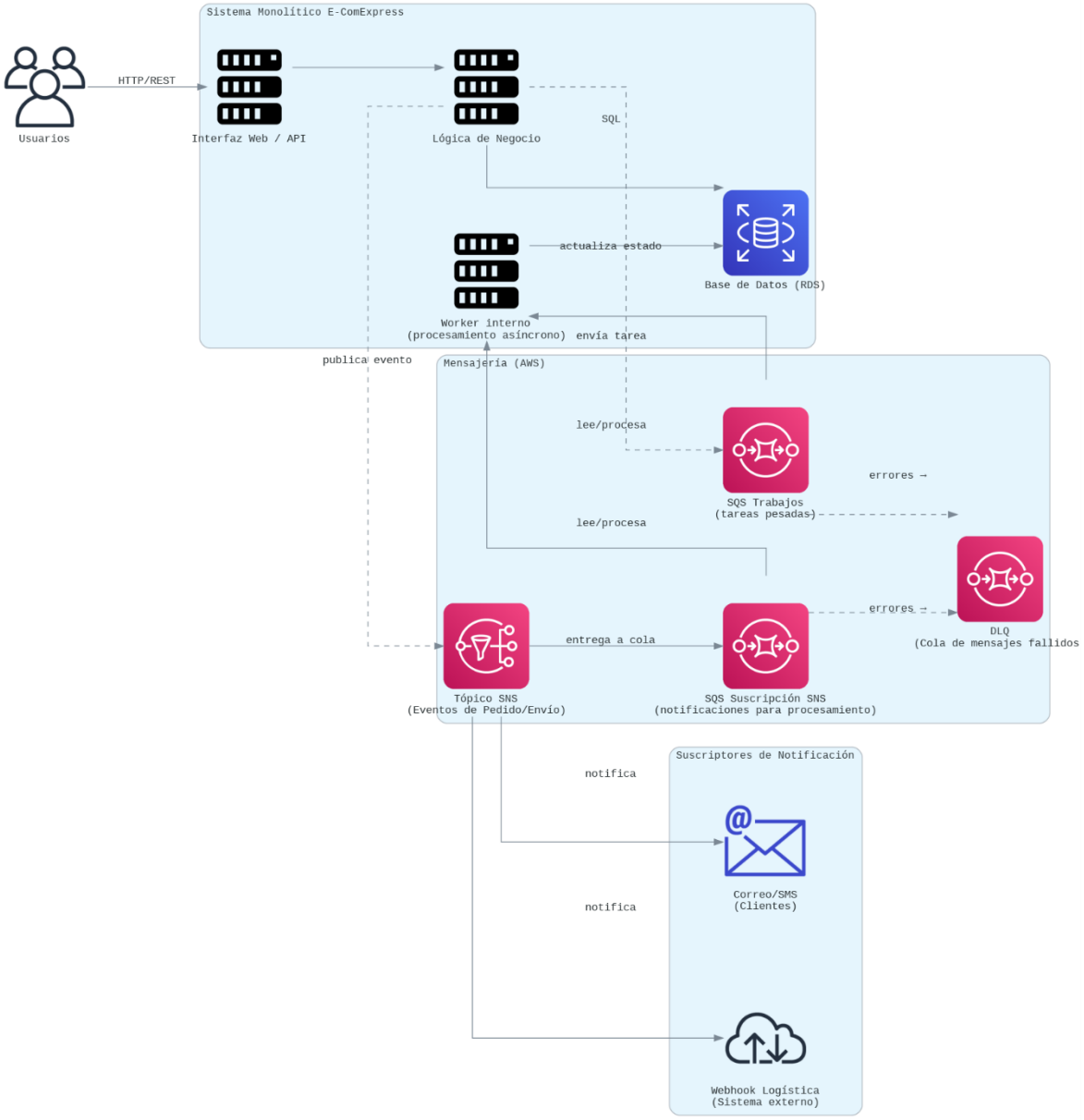
- **Acomplamiento fuerte y llamadas sincrónicas entre "procesamiento de pedidos" y "notificaciones de envío**: esto genera cuellos de botella. Cuando un módulo se satura, el otro se bloquea y se propagan errores y demoras en actualizar el estado del pedido. Esto degrada la experiencia del cliente.
- **Ausencia de un "buffer de mensajes" y reintentos:** un buffer de mensajes es una especie de "almacén temporal de mensajes" donde se guardan los mensajes que un sistema genera hasta que otro sistema o módulo pueda procesarlos. Sin una cola, los peaks de tráfico no se absorben. Si un consumidor cae, se pierden eventos o quedan atrasados. La mensajería asincrónica resuelve esto mediante reintentos y gestión de fallos.
- **Escalado limitado**: el modelo acoplado dificulta crecer según la demanda. Los sistemas de mensajería (SNS y SQS) facilitan la escalabilidad y el desacoplamiento.

Importancia de implementar servicios de mensajería

- **Desacoplar procesos**: que la comunicación sea por mensajes reduce dependencias directas entre módulos y esto facilita el mantenimiento y la evolución.
- **Tolerancia a fallos**: con SQS se tiene una especie de "colchón" entre productor y consumido. Además, permite reintentos, y tiene Dead Letter Queue (DLQ) es una cola especial donde se envían automáticamente mensajes que no pudieron ser procesados exitosamente después de un número máximo de intentos o que generaron errores recurrentes. Esta función ayuda a diagnosticar y corregir errores sin afectar el funcionamiento general del sistema. También se puede configurar la retención de mensajes, mejorando la resiliencia.
- **Difusión en tiempo real**: con SNS un módulo publica un evento y múltiples suscriptores reciben la notificación sin que el publicador conozca a cada destino.
- **Patrones complementarios**: con SNS (pub-sub, push) para alertas y SQS (cola, pull) para trabajo asincrónico a ritmo del consumidor, se puede mejorar tiempos de respuesta y confiabilidad.

2. Diseño de la solución

A continuación, se presenta un diagrama de la propuesta de solución para E-ComExpress.



Flujo de mensajes:

1) Usuario interactúa con la aplicación monolítica mediante HTTP/S.
2) Aplicación monolítica (procesamiento de pedidos):
   - Publica un evento "Pedido confirmado" en SNS – Tema: NotificacionesPedidos. Esto sigue el modelo "pub/sub".
3) SNS (tema):
   - Distribuye ese mensaje a múltiples suscriptores:
     - SQS-Cola "EventosLogística": para procesamiento

asíncrono.
    - Correo/SMS: avisos al cliente o equipo.
    - De manera opcional, se podría incorporar AWS Lambda para tareas inmediatas.

4) SQS – Cola "EventosLogística":
   - La cola retiene el mensaje de manera duradera y desacopla ritmos de producción/consumo (modelo pull). Los consumidores (módulo de logística dentro del monolito o un "worker") hacen poll, provesan y eliminan el mensaje.
5) Consumidores del monolito (módulo de logística/notificaciones):
   - Procesan el pedido: actualizan estado de envío, generan guía, notifican al cliente. Si el consumidor está saturado, SQS amortigua la carga hasta que pueda seguir procesando (asíncrono, backpressure).
6) Canales de notificación
   - SNS paralelamente empura (push) el mismo evento a email/SMS o a otros endpoints suscritos para notificaciones en tiempo real.

En el diagrama, **"Aplicación Monolítica"** está en un solo bloque, lo que refleja que todos los módulos internos están acoplados en la misma base de código.
El monolito se mantiene cohesionado pero desacoplado internamente por mensajería: menos bloqueos, mejor tolerancia a fallos y escalabilidad.

Justificación técnica de los componentes

- **SNS**: (Tema "NotificacionesPedidos")

Patón pub/sub para difundir un mismo evento a múltiples destinos (SQS, email, SMS, Lambda) sin dependencias punto a punto. Reduce acoplamiento y facilita evolución de integraciones.

- **SQS** (Cola "EventosLogística")

Amortigua picos y desacopla el ritmo productor/consumidor (pull). Permite reintentos, visibilidad, y retención (1 minuto a 14 días). Para orden estricto, usar FIFO; si prima el rendimiento, Estándar.

- **Consumidores del monolito (workers/módulos internos)**

Procesan mensajes asíncronamente, liberando el request original del pedido y evitando cuellos de botella en la lógica transaccional.

- **Políticas de acceso (IAM) y cifrado (KMS)**

Controlan quién publica en SNS y quién envía/lee en SQS; cifrado en reposo en ambos servicios. Mejora seguridad y cumplimiento.

- **Operación y confiabilidad**

CloudWatch para métricas (longitud de cola, tiempos), reintentos y DLQ para aislar mensajes con errores y evitar bloqueos del flujo.

SNS: notificaciones inmediatas y fan-out. SQS: procesamiento en segundo plano y control de flujo. La combinación mantiene la simplicidad monolítica con beneficios de resiliencia y escalabilidad típicos de sistemas desacoplados.

El diagrama muestra cómo un monolito puede mejorar su comunicación interna y externa usando SQS y SNS. SQS actúa como buffer de tareas para no sobrecargar la aplicación, mientras que SNS permite notificar en tiempo real a múltiples servicios o canales.

3. Propuesta de implementación

Configuración de Cola en SQS

1) Crear la cola

Consola AWS → SQS → "Create queue"

Elegir "Estándar" (alto rendimiento) o FIFO (orden estricto, evita duplicados) según el caso.

2) Políticas de acceso

Definir quién puede "enviar mensaje", "recibir mensaje", "borrar mensaje".
Habilitar cifrado en reposo con KMS en el caso que corresponda.

3) Parámetros clave

Definir tiempo de retención de mensajes.
Número máximo de mensajes por lectura.

4) Envío/recepción

La aplicación publica con la API, los consumidores hacen "poll", procesan y eliminan el mensaje.

5) Monitoreo y métricas
Métricas en CloudWatch y DLQ para mensajes fallidos/reintentos.

Creación de Tema/Tópico SNS:

1) Crear el tema

Consola AWS → SNS → "Create topic"

2) Políticas de acceso

Controlar quién puede publicar y suscribirse. Habilitar KMS.

3) Suscriptores

Añadir endpoints HTTP/HTTPS, email/SMS, SQS y/o Lambda. Cada suscriptor confirma su suscripción.

4) Publicación

La aplicación publica en el tema y SNS distribuye (push) el mismo mensaje a todos los suscriptores, incluyendo una cola SQS para procesamiento asíncrono.

5) Monitoreo/auditoría

Usar herramientas de AWS para verificar tráfico y cumplimiento de políticas, como CloudWatch, CloudTrail, IAM y KMS.

Ventajas de la solución

Tolerancia a fallos y resiliencia: permite reintentos y DLQ. Se evita cuellos de botella y pérdida de eventos.

Mejor integración de datos y desacoplamiento: pub/sub de SNS y colas de SQS separan productores y consumidores, facilitando evolución y mantenibilidad.

Optimización de tiempos de respuesta: la aplicación responde rápido y delega tareas a SQS. SNS notifica en tiempo real a múltiples destinos.

Escalabilidad: SQS permite "absorber" peaks de demandas y SNS distribuye a varios suscriptores.

4. Retroalimentación y Evaluación

Posibles desafíos o limitaciones

- **Dependencia de la configuración**: un error en las políticas IAM o en la suscripción puede provocar que mensajes no se entreguen correctamente. Esto afecta la trazabilidad y puede generar retrasos.
- **Gestión de mensajes fallidos**: si los consumidores no procesan los mensajes, estos se mantienen en la cola y podrían duplicarse.
- **Orden y duplicados**: las colas SQS estándar no garantizan orden estricto ni unicidad, lo que podría ser un problema en procesos críticos.
- **Latencia adicional**: al introducir SQS y SNS, el sistema añade un pequeño retardo en la comunicación.
- **Costos operativos**: si bien SQS y SNS son económicos, una mala configuración de los servicios puede incrementar costos de forma innecesaria.

Medidas de mitigación

- **Diseño de políticas de acceso claras (IAM**): aplicar principio de mínimo privilegio para que solo los servicios autorizados publiquen o consuman mensajes.
- **Uso de DLQ (Dead Letter Queue):** redirigir mensajes que fallan repetidamente, evitando bloqueos y facilitando su análisis posterior.
- **Elegir tipo de cola según necesidad**: usar FIFO en operaciones críticas que requieran orden y evitar duplicados (ej. pagos, actualizaciones de inventario).
- **Configurar métricas y alarmas en CloudWatch**: monitorear la longitud de la cola, tasa de errores y tiempos de entrega, generando alertas automáticas si algo se sale de rango.
- **Habilitar logging en SNS y SQS con CloudTrail**: auditar quién publica o consume mensajes, detectar usos indebidos o fallas en endpoints.
- **Diseñar reintentos con backoff exponencial**: configurar consumidores para reintentar procesamientos fallidos de forma escalonada, reduciendo presión sobre la cola.
- **Pruebas de carga y resiliencia**: simular picos de tráfico antes de pasar a producción, validando la capacidad de absorción de SQS y la difusión de SNS

5. Referencias

https://aws.amazon.com/es/sqs/
https://aws.amazon.com/es/sns/